

Analisis Sentimen Pengguna

Aplikasi Shopee Indonesia (Google Play Store)

Studi Kasus: Klasifikasi Sentimen Ulasan Pengguna Berbasis Machine Learning

RINGKASAN DOKUMEN

Jenis Dokumen	Laporan Teknis & Bisnis (Business-IT Report)
Objek Analisis	com.shopee.id Shopee Indonesia
Sumber Data	Google Play Store (via google-play-scraper)
Volume Data	3.000 ulasan pengguna
Periode Pengambilan	9 Juli 2026
Model Terbaik	Naive Bayes (F1-Score 91,50%)
Link Colab	https://drive.google.com/file/d/1AmFweTgOb4EZ7Y20qv-nu7VHlq4bC2p4/view?usp=sharing

Disusun oleh:

Aulia Febriyanti	1232002014
Chalimatus Sa'Diyah	1232002046
Najma Khonsa Tsabita	1232002090
Rega Saputra	1232002070

DAFTAR ISI

Ringkasan	3
Temuan Kunci	3
1. Latar Belakang & Tujuan	5
1.1 Latar Belakang.....	5
1.2 Tujuan.....	5
1.3 Ruang Lingkup	5
2. Metodologi	6
Alur Kerja (Pipeline)	6
2.1 Pengambilan Data (Data Collection)	6
2.2 Eksplorasi Data (EDA)	8
2.3 Pelabelan Sentimen.....	9
2.4 Text Preprocessing	10
2.5 Feature Engineering TF-IDF	15
2.6 Desain Eksperimen: 5 Model x 5 Skema Split	15
2.7 Metrik Evaluasi.....	19
3. Hasil & Analisis	21
3.1 Ringkasan Data Hasil Scraping	21
3.2 Distribusi Rating.....	21
3.3 Distribusi Sentimen	22
3.4 Ulasan dengan Engagement Tertinggi.....	22
3.5 Analisis Respons Developer	23
3.6 Tema Ulasan: Sebelum vs Sesudah Preprocessing.....	24
3.7 Hasil Evaluasi Model 25 Kombinasi Eksperimen	25
3.8 Ranking Global 10 Kombinasi Terbaik	27
3.9 Split Terbaik untuk Setiap Model	29
3.10 Konsistensi Model Rata-Rata di Seluruh Skema Split.....	29
3.11 Visualisasi Perbandingan Model x Split.....	30
4. Temuan Kunci	34
4.1 Temuan Teknis (Model & Data).....	34
4.2 Temuan Bisnis (Konten Ulasan).....	34
5. Risiko & Keterbatasan	35
6. Rekomendasi	36
6.1 Rekomendasi Teknis.....	36
6.2 Rekomendasi Bisnis	36
7. Rencana Tindak Lanjut	Error! Bookmark not defined.

Ringkasan

Laporan ini menyajikan hasil analisis sentimen terhadap 3.000 ulasan pengguna aplikasi Shopee Indonesia yang diambil dari Google Play Store. Tujuan utama proyek ini adalah membangun sistem klasifikasi otomatis yang mampu memilah ulasan pengguna ke dalam kategori positif dan negatif, sehingga tim produk dan customer experience dapat memantau persepsi pengguna secara lebih cepat dan terukur dibandingkan proses pembacaan manual.

Proses dikerjakan melalui alur end-to-end: pengambilan data (scraping), eksplorasi data (EDA), pembersihan teks berbahasa Indonesia (preprocessing dengan Sastrawi & indoNLP), representasi fitur (TF-IDF), hingga pelatihan dan evaluasi 5 algoritma machine learning pada 5 skema pembagian data latih/uji (total 25 kombinasi eksperimen).

3.000 Ulasan Dianalisis	91,50% F1-Score Terbaik	5 x 5 Model x Skema Split	73,2% Ulasan Dibalas Dev.
-----------------------------------	-----------------------------------	-------------------------------------	-------------------------------------

Model terbaik yang diperoleh adalah Naive Bayes dengan skema pembagian data 75% latih : 25% uji, mencapai akurasi 91,49% dan F1-Score 91,50%. Hasil ini konsisten Naive Bayes juga menjadi model dengan rata-rata performa terbaik di seluruh 5 skema split yang diuji, menunjukkan hasil yang stabil dan tidak bergantung pada satu konfigurasi split tertentu saja.

Temuan Kunci

- Distribusi rating sangat terpolarisasi: 1.591 ulasan bintang 5 dan 1.054 ulasan bintang 1, sementara bintang 2–4 jumlahnya jauh lebih sedikit.
- Setelah label netral (rating 3) dibuang, komposisi akhir adalah 1.690 ulasan positif dan 1.176 ulasan negatif (2.866 data siap model).
- Naive Bayes unggul secara konsisten di seluruh skema split, dengan Logistic Regression dan Random Forest di posisi berikutnya; KNN adalah model dengan performa paling lemah (F1-Score rata-rata hanya 54,5%).
- Tema keluhan dominan pada ulasan negatif: iklan yang mengganggu, aplikasi terasa berat/lambat, dan masalah pengiriman, sementara ulasan positif didominasi apresiasi terhadap kemudahan belanja dan kecepatan layanan.
- 73,2% ulasan sudah direspons oleh tim developer Shopee, menunjukkan tingkat keterlibatan customer service yang cukup baik, tapi masih menyisakan ruang perbaikan pada 26,8% ulasan yang belum dibalas.

Rekomendasi Utama

Gunakan model Naive Bayes (split 75:25) sebagai baseline produksi untuk sistem monitoring sentimen otomatis, sambil menindaklanjuti tema keluhan dominan (iklan & performa aplikasi) sebagai prioritas perbaikan produk jangka pendek.

1. Latar Belakang & Tujuan

1.1 Latar Belakang

Volume ulasan pengguna pada aplikasi berskala besar seperti Shopee terus bertambah setiap hari, membuat proses pemantauan sentimen secara manual menjadi tidak efisien dan rawan terlambat mendeteksi isu yang berkembang. Tim produk dan customer experience membutuhkan cara yang lebih cepat untuk mengetahui arah opini pengguna apakah cenderung positif atau negatif beserta topik apa yang paling sering dikeluhkan atau diapresiasi.

Proyek ini menjawab kebutuhan tersebut dengan membangun model klasifikasi sentimen berbasis machine learning yang dilatih dari data ulasan riil pengguna Shopee di Google Play Store, menggunakan pipeline pemrosesan teks Bahasa Indonesia yang disesuaikan dengan gaya bahasa sehari-hari (kata gaul, singkatan, typo umum di industri e-commerce).

1.2 Tujuan

1. Mengumpulkan data ulasan pengguna Shopee dalam volume yang representatif untuk pemodelan (3.000 ulasan terbaru).
2. Memahami pola dan distribusi sentimen pengguna melalui eksplorasi data (EDA).
3. Membangun pipeline pembersihan teks Bahasa Indonesia yang menangani slang, negasi, dan istilah domain e-commerce.
4. Melatih dan membandingkan performa 5 algoritma klasifikasi pada 5 skema pembagian data, untuk menemukan kombinasi model yang paling optimal dan stabil.
5. Menghasilkan rekomendasi bisnis dan teknis yang dapat langsung ditindaklanjuti oleh tim produk maupun tim data.

1.3 Ruang Lingkup

Lingkup proyek mencakup satu aplikasi (Shopee Indonesia, package ID com.shopee.id), berbahasa Indonesia, dengan horizon waktu ulasan terbaru (sort by newest) sebanyak 3.000 baris data. Analisis dibatasi pada klasifikasi sentimen biner (positif/negatif); ulasan dengan rating netral (bintang 3) dikeluarkan dari proses pelabelan karena tidak merepresentasikan polaritas yang jelas.

2. Metodologi

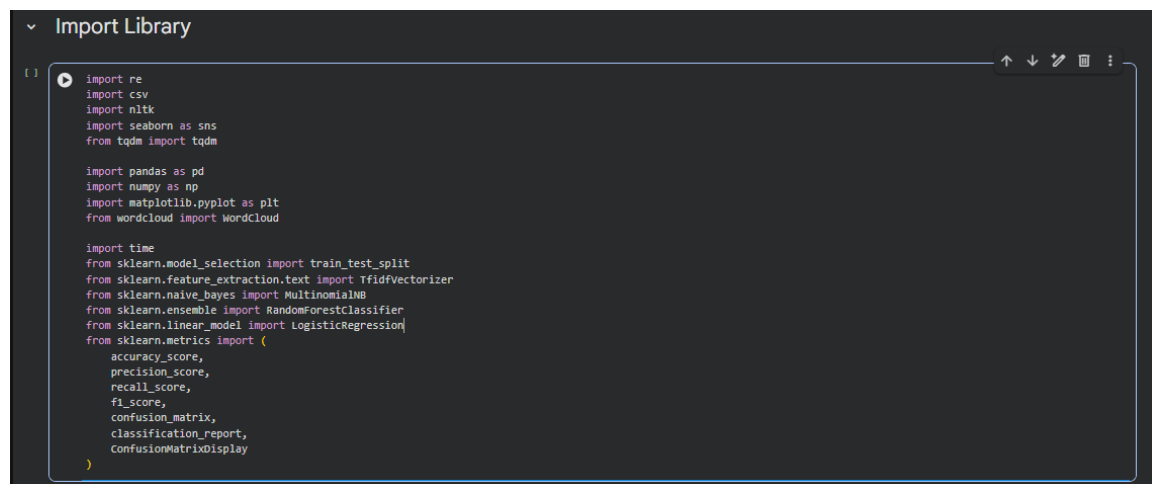
Pengerjaan proyek mengikuti alur kerja data science standar industri, mulai dari pengambilan data mentah hingga evaluasi model. Tahapannya sebagai berikut:

Alur Kerja (Pipeline)

1. Scraping Data
2. EDA
3. Text Preprocessing
4. Feature Engineering (TF-IDF)
5. Training 5 Model x 5 Split
6. Evaluasi & Ranking

2.1 Pengambilan Data (Data Collection)

Data diambil menggunakan library google-play-scraper dengan target 3.000 ulasan terbaru (sort=NEWEST) berbahasa Indonesia dari aplikasi Shopee Indonesia. Proses scraping dilakukan secara batch (200 ulasan per batch) dengan jeda 1 detik antar-batch untuk menghindari pembatasan akses (rate limiting) dari Google Play Store.



```
import re
import csv
import nltk
import seaborn as sns
from tqdm import tqdm

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from wordcloud import WordCloud

import time
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)
```

Tahap ini merupakan proses mengimpor berbagai library yang dibutuhkan selama penelitian. Library re, csv, dan nltk digunakan untuk proses text preprocessing, sedangkan pandas dan numpy digunakan untuk pengolahan data. Library matplotlib, seaborn, dan WordCloud dimanfaatkan untuk visualisasi data. Selanjutnya, train_test_split digunakan untuk membagi dataset menjadi data latih dan data uji, sedangkan TfidfVectorizer digunakan untuk mengubah data teks menjadi representasi numerik menggunakan metode TF-IDF. Penelitian ini menggunakan algoritma Multinomial Naïve Bayes, Random Forest Classifier, dan Logistic Regression sebagai model klasifikasi sentimen. Terakhir, beberapa fungsi pada sklearn.metrics digunakan untuk mengevaluasi performa model melalui metrik accuracy, precision, recall, F1-score, serta confusion matrix.

Scraping Review

Parameter scraping

```

[ ] app_id = "com.shopee.id" # Shopee Indonesia

    total_reviews = 3000
    lang = "id"
    country = "id"

    batch_size = 200
    sleep_time = 1

```

Scraping review shopee

```

[ ] !pip install google-play-scraper

from google_play_scraper import reviews, Sort

all_reviews = []
continuation_token = None

progress_bar = tqdm(
    total=total_reviews,
    desc="Mengambil Review Shopee"
)

while len(all_reviews) < total_reviews:

    remaining = total_reviews - len(all_reviews)
    current_batch_size = min(batch_size, remaining)

    result, continuation_token = reviews(
        app_id,
        lang=lang,
        country=country,
        sort=Sort.NEWEST,
        count=current_batch_size,
        continuation_token=continuation_token
    )

    if not result:
        print("Tidak ada review lagi.")
        break

    all_reviews.extend(result)
    progress_bar.update(len(result))

    if continuation_token is None:
        print("Review sudah habis.")
        break

    time.sleep(sleep_time)

progress_bar.close()

print(f"Jumlah review berhasil diambil: {len(all_reviews)}")

```

Kode ini digunakan untuk melakukan scraping data ulasan aplikasi Shopee dari Google Play Store menggunakan library google-play-scraper. Proses scraping dilakukan secara bertahap (batch) hingga jumlah ulasan yang diinginkan terpenuhi. Setiap hasil ulasan disimpan ke dalam variabel `all_reviews`, sementara `continuation_token` digunakan untuk mengambil data pada halaman berikutnya. Selain itu, `tqdm` digunakan untuk menampilkan progress pengambilan data, sedangkan `time.sleep()` memberikan jeda antar proses scraping agar permintaan ke server tidak dilakukan secara berlebihan. Setelah seluruh proses selesai, jumlah ulasan yang berhasil diperoleh akan ditampilkan dan digunakan sebagai dataset pada tahap analisis selanjutnya.

seleksi kolom

```

[ ] df = df[
    [
        "reviewId",
        "userName",
        "content",
        "score",
        "thumbsupcount",
        "reviewCreatedVersion",
        "at",
        "replyContent",
        "repliedAt",
        "appVersion"
    ]
]

```

Rename kolom

```
df = df.rename(columns={
    "reviewId": "review_id",
    "userName": "nama_pengguna",
    "content": "ulasan",
    "score": "rating",
    "thumbsUpCount": "jumlah_like",
    "reviewCreatedVersion": "versi_review",
    "at": "tanggal_review",
    "replyContent": "balasan_developer",
    "repliedAt": "tanggal_balasan",
    "appVersion": "versi_aplikasi"
})

df.head(10)
```

	review_id	nama_pengguna	ulasan	rating	jumlah_like	versi_review	tanggal_review	balasan_developer	tanggal_balasan	versi_aplikasi
0	5fbc95aa-bce4-4243-b0c1-bd9ebd0ca62f	Enbizi VF Indonesia	Kritik : Shopee Bantaeng jarang buka kadang ...	1	4	2.83.09	2026-07-09 10:32:33	None	NaT	2.83.09
1	877c30e-8ee5-4262-a929-68dc2d19c38	Rudy Doang	amanah ongkos kirim nya GK neko neko	5	0	3.77.26	2026-07-09 10:32:16	Hai Kak Rudy Doang, makasih ya untuk bintangny...	2026-07-09 15:24:01	3.77.26
2	64df451a-d7f1-4121-8213-37a933cca363	Sistra Putri	sebenemnya males belanja di shoppe lagi setela...	1	0	None	2026-07-09 10:31:38	Hai Kak Sistra Putri. Maaf ya buat Kakak ngga ...	2026-07-09 15:23:10	None

Setiap ulasan yang berhasil diambil memuat 11 atribut mentah (ID ulasan, nama pengguna, foto profil, isi ulasan, rating, jumlah like, versi aplikasi, tanggal ulasan, balasan developer, tanggal balasan, dan versi saat ulasan dibuat). Dari 11 kolom tersebut, 9 kolom relevan dipilih dan diganti namanya ke Bahasa Indonesia agar lebih mudah dipahami oleh pemangku kepentingan non-teknis (contoh: content → ulasan, score → rating, thumbsUpCount → jumlah_like).

2.2 Eksplorasi Data (EDA)

Setelah data terkumpul, dilakukan eksplorasi untuk memahami karakteristik data: distribusi rating, distribusi sentimen hasil pelabelan, ulasan dengan engagement tertinggi, serta tingkat respons developer terhadap ulasan pengguna.

Exploratory Data Analysis (EDA)

Head info describe

```
df.head()
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   review_id           3000 non-null   object
 1   nama_pengguna       3000 non-null   object
 2   ulasan              3000 non-null   object
 3   rating              3000 non-null   int64
 4   jumlah_like         3000 non-null   int64
 5   versi_review        2252 non-null   object
 6   tanggal_review      3000 non-null   datetime64[ns]
 7   balasan_developer   2191 non-null   object
 8   tanggal_balasan     2191 non-null   datetime64[ns]
 9   versi_aplikasi      2252 non-null   object
dtypes: datetime64[ns](2), int64(2), object(6)
memory usage: 234.5+ KB
```

	rating	jumlah_like	tanggal_review	tanggal_balasan
count	3000.000000	3000.000000	3000	2191
mean	3.350333	0.763667	2026-07-08 00:48:40.139666688	2026-05-30 22:34:24.190323968
min	1.000000	0.000000	2026-07-06 13:41:30	2017-03-27 08:28:42
25%	1.000000	0.000000	2026-07-07 07:25:17.5000000	2026-07-07 06:52:15.5000000
50%	5.000000	0.000000	2026-07-08 03:35:03	2026-07-08 02:42:07
75%	5.000000	0.000000	2026-07-08 15:48:22.249999872	2026-07-08 16:30:16.5000000
max	5.000000	1061.000000	2026-07-09 10:32:33	2026-07-09 15:24:01
std	1.865143	22.383033	NaN	NaN

Pada tahap Exploratory Data Analysis (EDA), dilakukan pemeriksaan awal terhadap dataset menggunakan fungsi head(), info(), dan describe(). Fungsi head() digunakan untuk menampilkan beberapa baris pertama dataset, info() untuk melihat struktur data seperti jumlah baris, tipe data, dan

nilai yang kosong (missing values), sedangkan `describe()` digunakan untuk menampilkan statistik deskriptif, seperti nilai rata-rata, nilai minimum, maksimum, median, dan standar deviasi. Tahap ini bertujuan untuk memahami karakteristik dataset sebelum dilakukan proses preprocessing dan pemodelan.

2.3 Pelabelan Sentimen

Label sentimen

```
[1]
#rating 4-5 = positif
#rating 1-2 = negatif
#rating 3 = dihapus

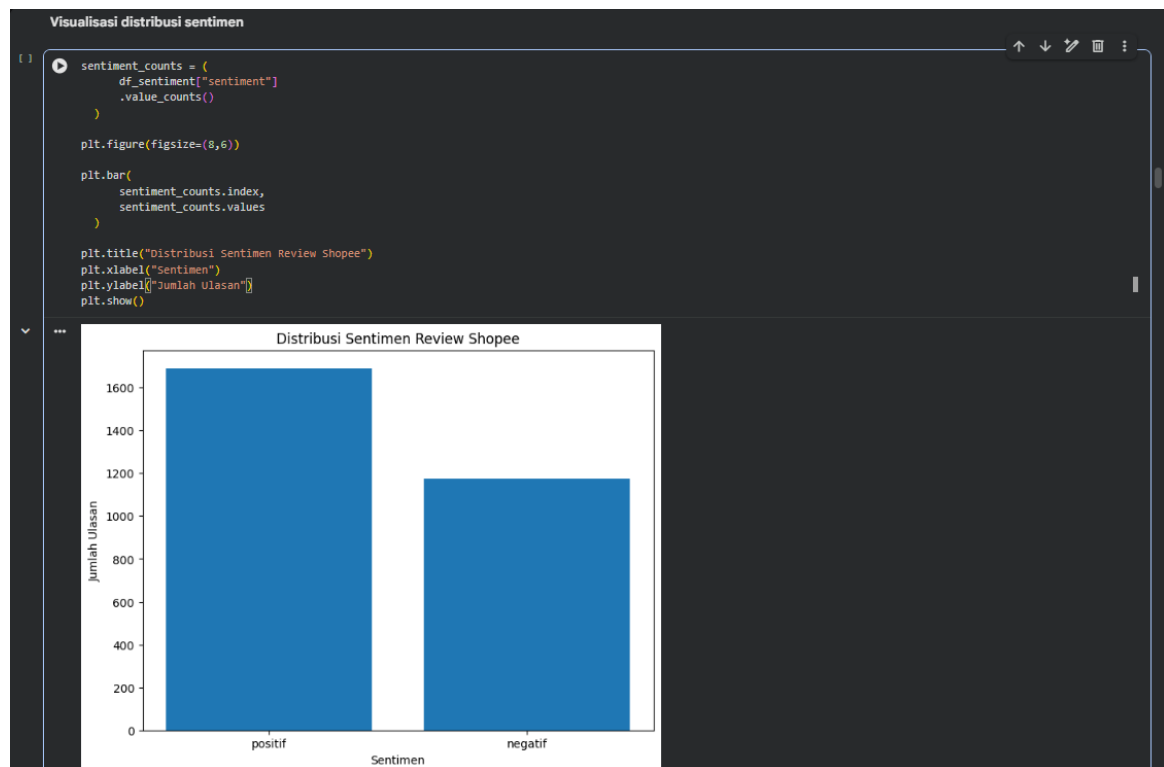
df_sentiment = df[df["rating"] != 3].copy()
df_sentiment["sentiment"] = np.where(
    df_sentiment["rating"] > 3,
    "positif",
    "negatif"
)
df_sentiment.head(10)
```

	review_id	nama_pengguna	ulasan	rating	jumlah_like	versi_review	tanggal_review	balasan_developer	tanggal_balasan	versi_aplikasi	sentiment
0	5f0c95aa-bca4-4243-b0c1-bd9ebd0ca62f	Enbizi VF Indonesia	Kritik : Shopee Bantaeng, jarang buka, kadang ...	1	4	2.83.09	2026-07-09 10:32:33	None	NaT	2.83.09	negatif
1	877c30e-8ee5-4262-a929-68dcf2419c38	Rudy Doang	amanah ongkos kirim nya GK neko neko	5	0	3.77.26	2026-07-09 10:32:16	Hai Kak Rudy Doang, makasih ya untuk bintangny...	2026-07-09 15:24:01	3.77.26	positif
2	64df451a-d7f1-4121-8213-37a933cca363	Sistra Putri	sebenarnya males belanja di shoppe lagi setela...	1	0	None	2026-07-09 10:31:38	Hai Kak Sistra Putri, Maaf ya buat Kakak ngga ...	2026-07-09 15:23:10	None	negatif

Label sentimen diturunkan dari nilai rating bintang yang diberikan pengguna, dengan aturan sebagai berikut:

- Rating 4–5 bintang → dilabeli “positif”
- Rating 1–2 bintang → dilabeli “negatif”
- Rating 3 bintang (netral) → dikeluarkan dari data pemodelan

Pendekatan ini adalah metode weak supervision yang umum digunakan dalam sentiment analysis berbasis rating, karena tidak memerlukan proses anotasi manual yang mahal dan memakan waktu, namun tetap menghasilkan label yang cukup akurat merepresentasikan polaritas ulasan.



2.4 Text Preprocessing

Ulasan pengguna aplikasi e-commerce umumnya ditulis dalam bahasa sehari-hari yang tidak baku bercampur singkatan, kata gaul, typo, dan istilah khas platform (ongkir, cod, resi, dsb). Pipeline pembersihan teks berikut dirancang khusus untuk menangani karakteristik ini:

1. Cleaning data

```
Cleaning data

[ ] df_model = df_sentiment.copy()

# Menghapus data duplikat
df_model = df_model.drop_duplicates()

# Menghapus review kosong
df_model = df_model[df_model["ulasan"].notnull()]

# Menghapus spasi berlebih
df_model["ulasan"] = df_model["ulasan"].str.strip()

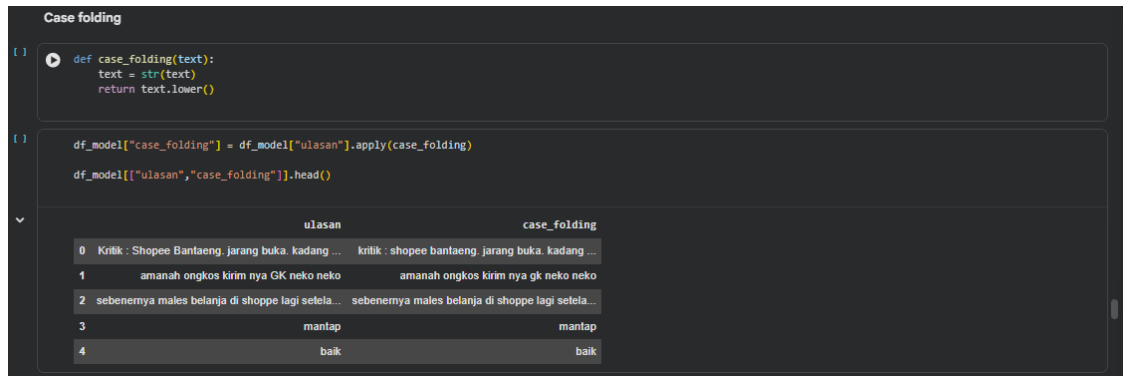
df_model[["ulasan"]].head()
```

	ulasan
0	Kritik : Shopee Bantaeng jarang buka. kadang ...
1	amanah ongkos kirim nya GK neko neko
2	sebenemnya males belanja di shoppe lagi setela...
3	mantap
4	baik

Pada tahap cleaning data, dilakukan pembersihan dataset agar kualitas data lebih baik sebelum diproses lebih lanjut. Proses ini meliputi pembuatan salinan dataset menggunakan `copy()`, penghapusan data duplikat dengan `drop_duplicates()`, penghapusan data yang memiliki nilai kosong pada kolom ulasan menggunakan `notnull()`, serta penghapusan spasi berlebih di awal dan akhir teks menggunakan `str.strip()`. Setelah proses pembersihan selesai, `head()` digunakan untuk menampilkan beberapa data pertama sebagai verifikasi bahwa data telah berhasil dibersihkan dan siap digunakan pada tahap preprocessing berikutnya.

Fungsi: mencegah model belajar dari data yang sama berulang kali (yang bisa membuat evaluasi bias/terlalu optimis) dan menghindari error saat teks kosong diproses oleh tahap berikutnya.

2. Case folding



```
def case_folding(text):
    text = str(text)
    return text.lower()

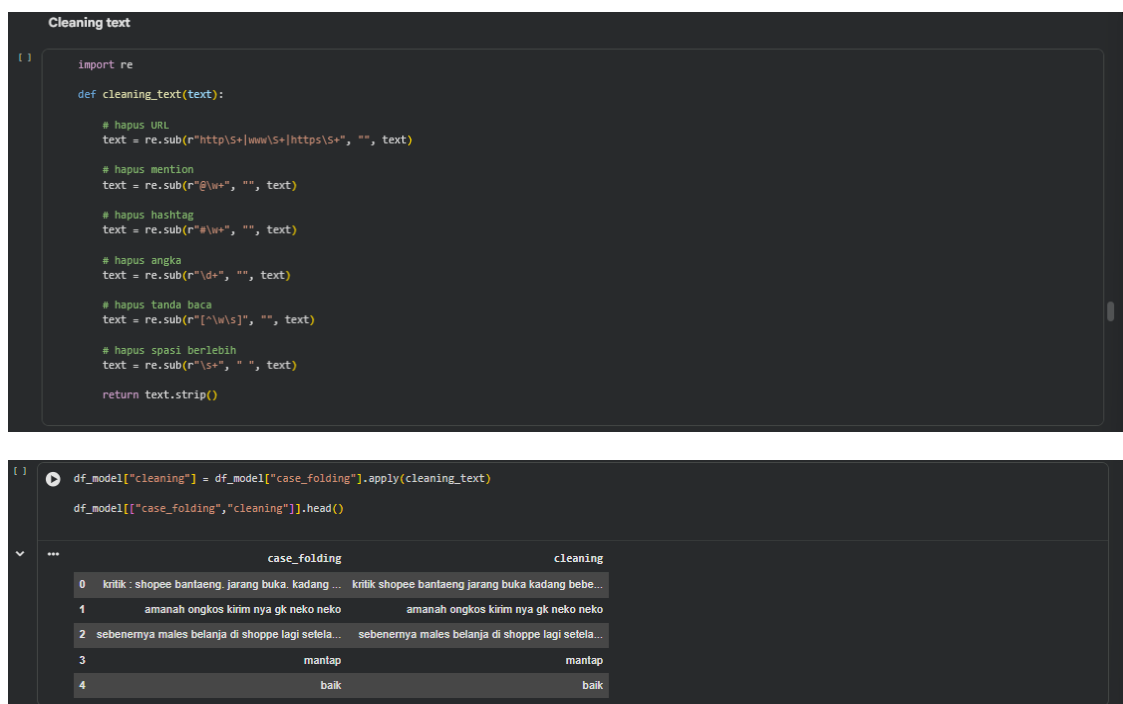
df_model["case_folding"] = df_model["ulasan"].apply(case_folding)
df_model[["ulasan", "case_folding"]].head()
```

	ulasan	case_folding
0	Kritik : Shopee Bantaeng, jarang buka, kadang ...	kritik : shopee bantaeng, jarang buka, kadang ...
1	amanah ongkos kirim nya GK neko neko	amanah ongkos kirim nya gk neko neko
2	sebenemnya males belanja di shoppe lagi setela...	sebenemnya males belanja di shoppe lagi setela...
3	mantap	mantap
4	baik	baik

Pada tahap case folding, seluruh teks pada kolom ulasan diubah menjadi huruf kecil (lowercase) menggunakan fungsi lower(). Proses ini dilakukan melalui fungsi case_folding() yang kemudian diterapkan ke setiap data menggunakan apply(). Tujuan dari tahap ini adalah menyeragamkan penulisan teks sehingga perbedaan huruf besar dan huruf kecil tidak memengaruhi proses analisis maupun pembentukan fitur pada tahap selanjutnya.

Fungsi: tanpa case folding, kata “Bagus” dan “bagus” akan dihitung TF-IDF sebagai dua fitur/kolom yang berbeda, padahal maknanya identic, ini akan memecah sinyal kata yang sama menjadi dua bagian lebih lemah.

3. Cleaning text



```
import re

def cleaning_text(text):
    # hapus URL
    text = re.sub(r"http[s+]{2}[www/s+]{2}[https/s+]", "", text)

    # hapus mention
    text = re.sub(r"@w+", "", text)

    # hapus hashtag
    text = re.sub(r"#w+", "", text)

    # hapus angka
    text = re.sub(r"d+", "", text)

    # hapus tanda baca
    text = re.sub(r"[^\w\s]", "", text)

    # hapus spasi berlebih
    text = re.sub(r"s+", " ", text)

    return text.strip()

df_model["cleaning"] = df_model["case_folding"].apply(cleaning_text)
df_model[["case_folding", "cleaning"]].head()
```

	case_folding	cleaning
0	kritik : shopee bantaeng, jarang buka, kadang ...	kritik shopee bantaeng jarang buka kadang bebe...
1	amanah ongkos kirim nya gk neko neko	amanah ongkos kirim nya gk neko neko
2	sebenemnya males belanja di shoppe lagi setela...	sebenemnya males belanja di shoppe lagi setela...
3	mantap	mantap
4	baik	baik

Pada tahap cleaning text, dilakukan pembersihan isi teks menggunakan regular expression (re) untuk menghilangkan elemen-elemen yang tidak diperlukan dalam analisis sentimen. Proses ini meliputi penghapusan URL, mention (@), hashtag (#), angka, tanda baca atau karakter khusus, serta spasi berlebih. Setelah seluruh proses selesai, fungsi strip() digunakan untuk menghapus spasi di awal dan akhir teks sehingga data ulasan menjadi lebih bersih dan siap diproses pada tahap preprocessing berikutnya.

Fungsi: URL, mention, dan angka umumnya adalah noise yang tidak membawa informasi sentimen, sehingga menghapusnya membuat representasi TF-IDF lebih fokus pada kata-kata yang benar-benar bermakna.

4. Normalisasi slang

```

Setup library (Sastrawi & indonLP)

!pip install Sastrawi indonLP -q

289.7/289.7 kB 6.8 MB/s eta 0:00:00
121.9/121.9 kB 6.5 MB/s eta 0:00:00

Normalisasi kata gaul/slang
Pakai library indonLP (kamus Colloquial Indonesian Lexicon bawaan library) untuk mengubah kata gaul/alay menjadi kata baku, sekaligus
merapikan huruf yang diulang-ulang seperti bagusss jadi bagus

from indonLP.preprocessing import replace_word_elongation, replace_slang, pipeline

# pipeline: rapikan huruf berulang dulu, baru ganti kata slang -> baku
normalize_pipeline = pipeline([replace_word_elongation, replace_slang])

def normalisasi_teks(text):
    text = normalize_pipeline(text)
    # rapikan sisa typo/istilah domain yang belum dikenali indonLP
    text = normalisasi_custom(text)
    return text

import re

# Tambahkan/entri baru di sini kalau masih menemukan typo/istilah lain
kamus_tambahan = {
    "eror": "error",
    "erorr": "error",
    "gabisa": "tidak bisa",
    "ga bisa": "tidak bisa",
    "gakbisa": "tidak bisa",
    "cs": "customer service",
    "hp": "handphone",
    "wa": "whatsapp",
    "ol": "online",
    "gopay": "gopay",
    "cod": "cash on delivery",
    "resi": "resi",
    "ongkir": "ongkos kirim",
    "chat": "chat",
    "seller": "penjual",
    "buyer": "pembeli",
}

def normalisasi_custom(text):
    for kata_salah, kata_benar in kamus_tambahan.items():
        text = re.sub(rf"(?!\b{re.escape(kata_salah)}\b)", kata_benar, text)
    return text

df_model["normalisasi"] = df_model["cleaning"].apply(normalisasi_teks)
df_model[["cleaning", "normalisasi"]].head()

```

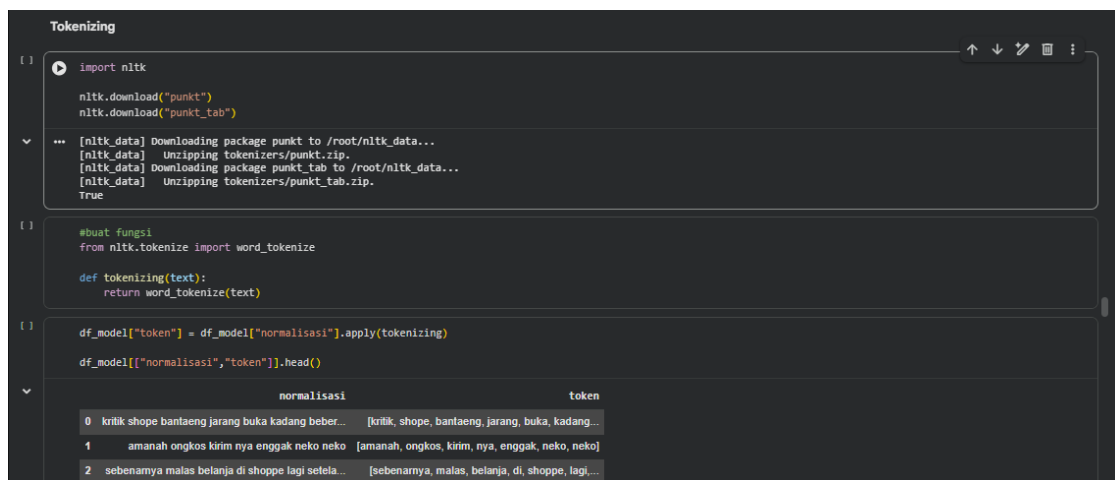
	cleaning	normalisasi
0	kritik shopee bantaeng jarang buka kadang bebe...	kritik shope bantaeng jarang buka kadang beber...
1	amanah ongkos kirim nya gk neko neko	amanah ongkos kirim nya enggak neko neko
2	sebenemnya males belanja di shopee lagi seela...	sebenamya malas belanja di shopee lagi setela...
3	mantap	mantap
4	baik	baik

Pada tahap normalisasi kata gaul/slang, digunakan library indonLP untuk mengubah kata-kata tidak baku, kata gaul, maupun huruf yang ditulis berulang menjadi bentuk kata baku. Proses ini dilakukan

menggunakan pipeline yang terdiri dari `replace_word_elongation` untuk merapikan huruf berulang dan `replace_slang` untuk mengganti kata slang ke bentuk bakunya. Selain itu, ditambahkan fungsi `normalisasi_custom()` untuk menyesuaikan kata atau istilah tertentu yang belum tersedia pada kamus `indoNLP`, sehingga hasil normalisasi menjadi lebih akurat sebelum memasuki tahap preprocessing berikutnya. Ditambah kamus koreksi khusus istilah e-commerce (`eror`→`error`, `gabisa`→`tidak bisa`, `cs`→`customer service`, `ongkir`→`ongkos kirim`, dll).

Fungsi: tanpa normalisasi, variasi ejaan seperti “gpp”, “gapapa”, dan “tidak apa-apa” akan dihitung sebagai tiga fitur terpisah oleh model, padahal maknanya sama, normalisasi menyatukan sinyal tersebut menjadi satu fitur yang lebih kuat secara statistik.

5. Tokenizing



```

import nltk

nltk.download("punkt")
nltk.download("punkt_tab")

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
True

#buat fungsi
from nltk.tokenize import word_tokenize

def tokenizing(text):
    return word_tokenize(text)

df_model["token"] = df_model["normalisasi"].apply(tokenizing)

df_model[["normalisasi", "token"]].head()

```

	normalisasi	token
0	kritik Shope bantaeng jarang buka kadang beber...	[kritik, Shope, bantaeng, jarang, buka, kadang, beber...]
1	amanah ongkos kirim nya enggak neko neko	[amanah, ongkos, kirim, nya, enggak, neko, neko]
2	sebenarnya malas belanja di shoppe lagi setela...	[sebenarnya, malas, belanja, di, shoppe, lagi, setela...]

Pada tahap tokenizing, teks hasil normalisasi dipecah menjadi kata-kata tunggal (token) menggunakan fungsi `word_tokenize()` dari library NLTK. Sebelum proses tokenisasi dilakukan, package `punkt` diunduh sebagai model yang digunakan oleh NLTK untuk memisahkan teks menjadi token. Hasil tokenisasi kemudian disimpan pada kolom `token`, sehingga setiap ulasan berubah menjadi daftar kata yang siap digunakan pada tahap preprocessing selanjutnya, seperti `stopword removal` dan `stemming`.

Fungsi: TF-IDF bekerja pada level kata (bukan kalimat utuh), sehingga tokenizing adalah prasyarat wajib sebelum kata dapat dihitung frekuensinya satu per satu.

6. Stopword removal

```

from Sastrawi.StopwordRemover.StopwordRemoverFactory import StopwordRemoverFactory

stopword_factory = StopwordRemoverFactory()
stopword_default = set(stopword_factory.get_stop_words())

stopword_tambahan = {
    "nya", "sih", "dong", "deh", "dech", "kok", "toh", "lah", "ya", "yah",
    "aja", "saja", "kayak", "kaya", "banget", "bgt", "pas", "kadang",
    "terus", "juga", "sama", "sekali", "gitu", "gini", "biar", "tuh",
    "eh", "nih", "min", "admin", "gan", "bang", "kak", "cuy", "bro", "sis",
    "doang", "cuma", "hanya", "utk", "untuk", "dgn", "dengan", "yaitu",
    "yakni", "woy", "wah", "loh", "lho", "yg", "tp", "dr", "krn", "spy",
}

# daftar Sastrawi sendiri sudah memasukkan kata negasi seperti "tidak", "nggak", "belum" sebagai stopword! Kalau dibiarkan, "tidak bagus" akan berubah jadi "bagus" saja
# sehingga makna/label sentimen jadi terbalik. Maka kata negasi WAJIB dikecualikan (whitelist), baik dari daftar bawaan maupun tambahan.
negasi_whitelist = {
    "tidak", "tak", "gak", "ga", "enggak", "nggak", "bukan",
    "belum", "blm", "kurang", "jangan", "tanpa",
}

stopword_id = (stopword_default.union(stopword_tambahan)) - negasi_whitelist
print("Jumlah stopword Sastrawi (default) :", len(stopword_default))
print("Jumlah stopword tambahan :", len(stopword_tambahan))
print("Jumlah kata negasi dikecualikan :", len(negasi_whitelist))
print("Jumlah stopword total dipakai :", len(stopword_id))

```

Jumlah stopword Sastrawi (default) : 123
 Jumlah stopword tambahan : 54
 Jumlah kata negasi dikecualikan : 12
 Jumlah stopword total dipakai : 164

```

Stop word removal

def remove_stopword(tokens):
    hasil = []
    for word in tokens:
        if word not in stopword_id:
            hasil.append(word)
    return hasil

df_model["stopword"] = (
    df_model["token"]
    .apply(remove_stopword)
)
df_model[["token", "stopword"]].head()

```

	token	stopword
0	[kritik, shope, bantaeng, jarang, buka, kadang ...	[kritik, shope, bantaeng, jarang, buka, bebara...
1	[amanah, ongkos, kirim, nya, enggak, neko, neko]	[amanah, ongkos, kirim, enggak, neko, neko]
2	[sebenarnya, malas, belanja, di, shopee, lagi...	[sebenarnya, malas, belanja, shopee, kendala, ...

Menghapus kata-kata umum yang tidak bermakna signifikan, menggunakan kombinasi 123 stopword baku dari library Sastrawi ditambah 54 stopword gaul tambahan (nya, sih, dong, banget, dll), dengan pengecualian eksplisit untuk 12 kata negasi (tidak, gak, belum, bukan, dll) agar makna sentimen tidak terbalik.

Fungsi: mengurangi dimensi fitur TF-IDF secara signifikan (kata umum seperti “yang” bisa muncul di ribuan ulasan tanpa membedakan sentimen), sekaligus whitelist negasi menjaga agar makna kalimat seperti “tidak puas” tidak berubah menjadi netral/positif akibat kata “tidak” ikut terhapus.

Daftar stopword bawaan Sastrawi ternyata memasukkan kata negasi (“tidak”, “nggak”, “belum”) sebagai stopword. Bila dibiarkan, frasa “tidak bagus” akan tereduksi menjadi “bagus” setelah stopword removal membalik makna sentimen dari negatif menjadi seolah positif. Kata-kata negasi ini secara eksplisit dikecualikan (whitelist) dari proses penghapusan stopword.

7. Stemming

```

Setup Stemmer (Sastrawi)

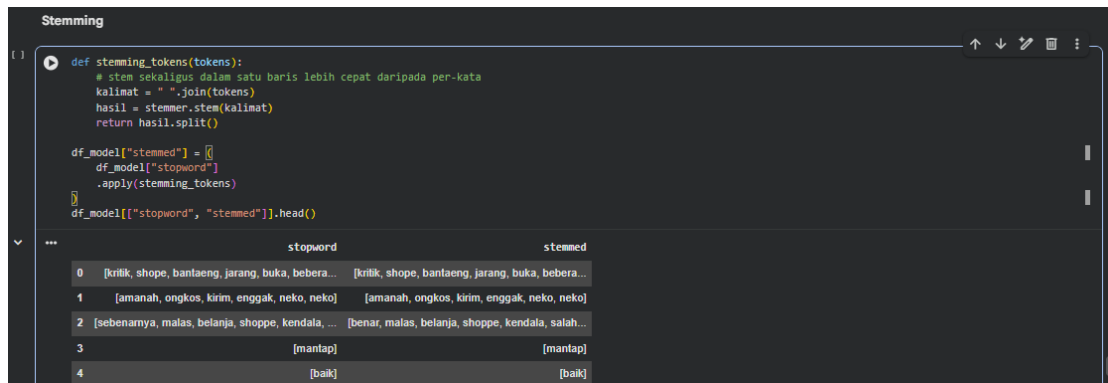
# Stemmer Bahasa Indonesia pakai Sastrawi
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

stemmer_factory = StemmerFactory()
stemmer = stemmer_factory.create_stemmer()

# contoh
print(stemmer.stem("pengiriman barang lambat dan tidak dibalas"))

```

kirim barang lambat dan tidak balas



```
def stemming_tokens(tokens):
    # stem sekaligus dalam satu baris lebih cepat daripada per-kata
    kalimat = " ".join(tokens)
    hasil = stemmer.stem(kalimat)
    return hasil.split()

df_model["stemmed"] = []
df_model["stopword"]
    .apply(stemming_tokens)

df_model[["stopword", "stemmed"]].head()
```

	stopword	stemmed
0	[kritik, shope, bantaeng, jarang, buka, bebara...	[kritik, shope, bantaeng, jarang, buka, bebara...
1	[amanah, ongkos, kirim, enggak, neko, neko]	[amanah, ongkos, kirim, enggak, neko, neko]
2	[sebenarnya, malas, belanja, shope, kendala, ...]	[benar, malas, belanja, shope, kendala, salah...
3	[mantap]	[mantap]
4	[baik]	[baik]

Mengembalikan kata berimbuhan ke bentuk dasarnya menggunakan Sastrawi Stemmer (contoh: “pengiriman” → “kirim”, “diperbaiki” → “baik”).

Fungsi: menyatukan kata berimbuhan seperti “mengirim”, “dikirim”, dan “pengiriman” menjadi satu bentuk dasar “kirim”, sehingga model tidak memperlakukan ketiganya sebagai fitur terpisah ini memperkuat sinyal statistik kata tersebut sekaligus mengurangi total dimensi fitur.

2.5 Feature Engineering TF-IDF

Teks hasil preprocessing diubah menjadi representasi numerik menggunakan TF-IDF (Term Frequency–Inverse Document Frequency), teknik standar dalam pemrosesan teks yang memberi bobot lebih tinggi pada kata-kata yang sering muncul dalam satu ulasan namun jarang muncul di ulasan lain sehingga kata-kata yang benar-benar informatif untuk membedakan sentimen mendapat bobot lebih besar dibanding kata umum.

2.6 Desain Eksperimen: 5 Model x 5 Skema Split

Untuk memastikan model terbaik ditemukan secara sistematis (bukan kebetulan dari satu konfigurasi saja), setiap kombinasi berikut diuji secara menyeluruh, total 25 kombinasi eksperimen:

Algoritma	Karakteristik Singkat
Naive Bayes (MultinomialNB)	Probabilistik, sangat efisien untuk data teks/TF-IDF
Decision Tree	Berbasis aturan hierarkis, mudah diinterpretasi
K-Nearest Neighbors (KNN)	Berbasis kedekatan jarak antar-dokumen
Random Forest	Ensemble dari banyak decision tree
Logistic Regression	Model linear klasik untuk klasifikasi biner
Skema Split (Latih:Uji)	Proporsi Data Uji
80 : 20	20%
75 : 25	25%
70 : 30	30%
65 : 35	35%
60 : 40	40%

```
Import Library

# Manipulasi Data
import pandas as pd
import numpy as np

# Train-Test Split
from sklearn.model_selection import train_test_split

# TF-IDF
from sklearn.feature_extraction.text import TfidfVectorizer

# Label Encoding
from sklearn.preprocessing import LabelEncoder

# Model Machine Learning
from sklearn.naive_bayes import MultinomialNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression

# Evaluasi
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# Visualisasi (dipakai nanti)
import matplotlib.pyplot as plt
import seaborn as sns
```

Library pandas dan numpy digunakan untuk manipulasi data, train_test_split untuk membagi data latih dan data uji, serta TfidfVectorizer untuk mengubah data teks menjadi representasi numerik menggunakan metode TF-IDF. Selain itu, LabelEncoder digunakan untuk mengubah label sentimen ke dalam bentuk numerik. Penelitian ini menggunakan algoritma Multinomial Naïve Bayes, Decision Tree, K-Nearest Neighbor (KNN), Random Forest, dan Logistic Regression sebagai model klasifikasi. Terakhir, beberapa fungsi pada sklearn.metrics digunakan untuk mengevaluasi performa model berdasarkan nilai accuracy, precision, recall, F1-score, confusion matrix, dan classification report, sedangkan Matplotlib dan Seaborn digunakan untuk visualisasi hasil analisis.

```
Feature & target

#memisahkan fitur dan target

# Feature (teks hasil preprocessing)
X = df_model["review_clean"]

# Target (label sentimen)
y = df_model["sentiment"]

# Menampilkan jumlah data
print("Jumlah Data :", len(X))
print("Jumlah Label:", len(y))

Jumlah Data : 2866
Jumlah Label: 2866
```

Pada tahap ini dilakukan pemisahan antara fitur (feature) dan target (label) yang akan digunakan dalam proses klasifikasi. Kolom review_clean digunakan sebagai fitur (X) karena berisi teks ulasan yang telah melalui tahap preprocessing, sedangkan kolom sentiment digunakan sebagai target (y) yang berisi label sentimen. Selanjutnya, jumlah data pada fitur dan target ditampilkan untuk memastikan bahwa keduanya memiliki jumlah data yang sama sehingga siap digunakan pada tahap pembagian data (train-test split) dan pelatihan model machine learning.

```

Label encoding

[]
#karena xgboost dan ANN membutuhkan label numerik
label_encoder = LabelEncoder()

y_encode = label_encoder.fit_transform(y)

# Melihat hasil encoding
print("Label Asli :", label_encoder.classes_)
print("Hasil Encoding :", np.unique(y_encode))

Label Asli : ['negatif' 'positif']
Hasil Encoding : [0 1]

```

Pada tahap label encoding, label sentimen yang masih berbentuk teks diubah menjadi nilai numerik menggunakan LabelEncoder dari Scikit-learn. Proses ini diperlukan agar label dapat diproses oleh algoritma machine learning yang hanya menerima input numerik. Pada penelitian ini, label "negatif" dikodekan menjadi 0, sedangkan label "positif" dikodekan menjadi 1. Hasil encoding kemudian digunakan sebagai target pada proses pelatihan dan pengujian model klasifikasi.

```

Loop Train - Test split & TD - IDF

[]
split_list = {
    "80:20":0.20,
    "75:25":0.25,
    "70:30":0.30,
    "65:35":0.35,
    "60:40":0.40
}
hasil = []

#membuatnya dalam bentuk dictionary agar tidak manggil berulang-ulang
models = {
    "Naive Bayes": MultinomialNB(),
    "Decision Tree": DecisionTreeClassifier(
        random_state=42
    ),
    "KNN": KNeighborsClassifier(
        n_neighbors=5
    ),
    "Random Forest": RandomForestClassifier(
        random_state=42
    ),
    "Logistic Regression": LogisticRegression(
        max_iter=1000,
        random_state=42
    )
}

for split_name, test_size in split_list.items():
    print("="*60)
    print(f"TRAIN : TEST = {split_name}")
    print("="*60)

```

Pada tahap ini dilakukan proses train-test split dengan lima skenario pembagian data, yaitu 80:20, 75:25, 70:30, 65:35, dan 60:40, untuk membandingkan performa model pada proporsi data latih dan data uji yang berbeda. Selain itu, dibentuk sebuah dictionary yang berisi algoritma klasifikasi, yaitu Multinomial Naïve Bayes, Decision Tree, K-Nearest Neighbor (KNN), Random Forest, dan Logistic Regression, sehingga setiap model dapat dijalankan secara otomatis melalui proses perulangan (loop). Pendekatan ini membuat proses pelatihan dan evaluasi menjadi lebih efisien serta memudahkan perbandingan hasil kinerja setiap model pada masing-masing skenario pembagian data.

```

# TRAIN TEST SPLIT
X_train_text, X_test_text, y_train, y_test = train_test_split(
    X,
    y,
    test_size=test_size,
    random_state=42,
    stratify=y
)

```

Pada tahap ini, dataset dibagi menjadi data latih (training data) dan data uji (testing data) menggunakan fungsi `train_test_split()`. Variabel X yang berisi fitur dan y yang berisi label sentimen dipisahkan berdasarkan proporsi `test_size` yang telah ditentukan pada setiap skenario pengujian. Parameter

random_state=42 digunakan agar hasil pembagian data tetap konsisten pada setiap proses, sedangkan stratify=y memastikan distribusi kelas sentimen pada data latih dan data uji tetap seimbang sehingga proses pelatihan dan evaluasi model dapat dilakukan secara lebih representatif.

```
# MENAMPILKAN JUMLAH DATA
print("Jumlah Data Latih :", len(X_train_text))
print("Jumlah Data Uji   :", len(X_test_text))

#TF - IDF
tfidf = TfidfVectorizer()

X_train = tfidf.fit_transform(X_train_text);
X_test  = tfidf.transform(X_test_text);

# MENAMPILKAN UKURAN TF-IDF
print("Ukuran TF-IDF train :", X_train.shape)
print("Ukuran TF-IDF Test  :", X_test.shape)

print("-"*60)

for model_name, model in models.items():
```

Pada tahap ini, data latih dan data uji terlebih dahulu ditampilkan jumlahnya untuk memastikan proses train-test split telah berhasil. Selanjutnya, digunakan TfidfVectorizer untuk mengubah data teks menjadi representasi numerik menggunakan metode Term Frequency–Inverse Document Frequency (TF-IDF). Proses fit_transform() diterapkan pada data latih untuk membentuk kosakata (vocabulary) dan menghitung bobot TF-IDF, sedangkan transform() digunakan pada data uji agar menggunakan kosakata yang sama dengan data latih. Terakhir, ukuran matriks TF-IDF ditampilkan untuk memverifikasi dimensi fitur yang akan digunakan pada proses pelatihan model klasifikasi.

```
# Training Model
model.fit(
    X_train,
    y_train
)

# Prediksi
y_pred = model.predict(
    X_test
)

# Evaluasi
accuracy = accuracy_score(
    y_test,
    y_pred
)

precision = precision_score(
    y_test,
    y_pred,
    average="weighted"
)

recall = recall_score(
    y_test,
    y_pred,
    average="weighted"
)

f1 = f1_score(
    y_test,
    y_pred,
    average="weighted"
)
```

Pada tahap ini, setiap algoritma machine learning dilatih menggunakan data latih melalui fungsi fit(). Setelah proses pelatihan selesai, model melakukan prediksi terhadap data uji menggunakan fungsi predict(). Selanjutnya, performa model dievaluasi menggunakan beberapa metrik, yaitu accuracy untuk mengukur ketepatan prediksi secara keseluruhan, precision untuk mengukur ketepatan prediksi pada setiap kelas, recall untuk mengukur kemampuan model dalam mengenali seluruh data pada masing-masing kelas, serta F1-score sebagai rata-rata harmonis antara precision dan recall. Pada metrik precision, recall, dan F1-score digunakan parameter average="weighted" agar hasil evaluasi mempertimbangkan proporsi jumlah data pada setiap kelas.

```

# Simpan Hasil
hasil.append({
    "Split": split_name,
    "Model": model_name,
    "Accuracy": accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1-Score": f1
})

print(f"{model_name} selesai.")

```

```

***
=====
TRAIN : TEST = 80:20
=====
Jumlah Data Latih : 2292
Jumlah Data Uji : 574
Ukuran TF-IDF Train : (2292, 2818)
Ukuran TF-IDF Test : (574, 2818)
=====
Naive Bayes selesai.
Decision Tree selesai.
KNN selesai.
Random Forest selesai.
Logistic Regression selesai.
=====
TRAIN : TEST = 75:25
=====
Jumlah Data Latih : 2149
Jumlah Data Uji : 717
Ukuran TF-IDF Train : (2149, 2730)
Ukuran TF-IDF Test : (717, 2730)
=====
Naive Bayes selesai.
Decision Tree selesai.
KNN selesai.
Random Forest selesai.
Logistic Regression selesai.
=====
TRAIN : TEST = 70:30
=====
Jumlah Data Latih : 2006
Jumlah Data Uji : 860
Ukuran TF-IDF Train : (2006, 2622)
Ukuran TF-IDF Test : (860, 2622)
=====
Naive Bayes selesai.
Decision Tree selesai.
KNN selesai.
Random Forest selesai.
Logistic Regression selesai.

```

Pada tahap ini, hasil evaluasi dari setiap model disimpan ke dalam sebuah list bernama hasil menggunakan fungsi append(). Data yang disimpan meliputi skenario pembagian data (split), nama model, serta nilai accuracy, precision, recall, dan F1-score. Selanjutnya, program menampilkan informasi proses pelatihan, seperti jumlah data latih dan data uji, ukuran matriks TF-IDF, serta status penyelesaian setiap algoritma. Penyimpanan hasil ini bertujuan untuk memudahkan proses perbandingan performa seluruh model pada setiap skenario train-test split dan digunakan sebagai dasar analisis hasil penelitian.

2.7 Metrik Evaluasi

Setiap kombinasi model dan skema split dievaluasi menggunakan empat metrik standar klasifikasi, dengan rata-rata berbobot (weighted average) untuk mengakomodasi ketidakseimbangan jumlah data antar-kelas (positif lebih banyak daripada negatif):

- Accuracy

Proporsi prediksi yang benar dari keseluruhan data uji.

Fungsi: Kapan diandalkan: paling representatif ketika kelas data seimbang (seperti pada kasus ini, 59:41); pada data yang timpang, accuracy tinggi bisa menyesatkan karena model bisa saja hanya menebak kelas mayoritas.

- Precision

Dari semua ulasan yang diprediksi positif/negatif, berapa persen yang benar-benar sesuai.

Fungsi: Kapan diprioritaskan: penting ketika biaya salah memprediksi “positif” (false positive) tinggi misalnya bila hasil model dipakai untuk memutuskan ulasan mana yang TIDAK perlu ditindaklanjuti tim CS.

- Recall

Dari semua ulasan yang sebenarnya positif/negatif, berapa persen yang berhasil terdeteksi model.

Fungsi: Kapan diprioritaskan: penting ketika biaya GAGAL mendeteksi kasus “negatif” (false negative) tinggi misalnya bila model dipakai untuk sistem deteksi dini keluhan yang wajib ditangani.

- F1-Score

Rata-rata harmonik precision dan recall, dipakai sebagai metrik utama untuk perankingan karena menyeimbangkan kedua sisi kesalahan (false positive dan false negative).

Fungsi: Fungsi: karena precision dan recall sering saling bertukar (trade-off), F1-Score dipilih sebagai metrik ranking tunggal supaya perbandingan antar 25 kombinasi model×split tetap adil dan tidak bias ke salah satu sisi kesalahan.

3. Hasil & Analisis

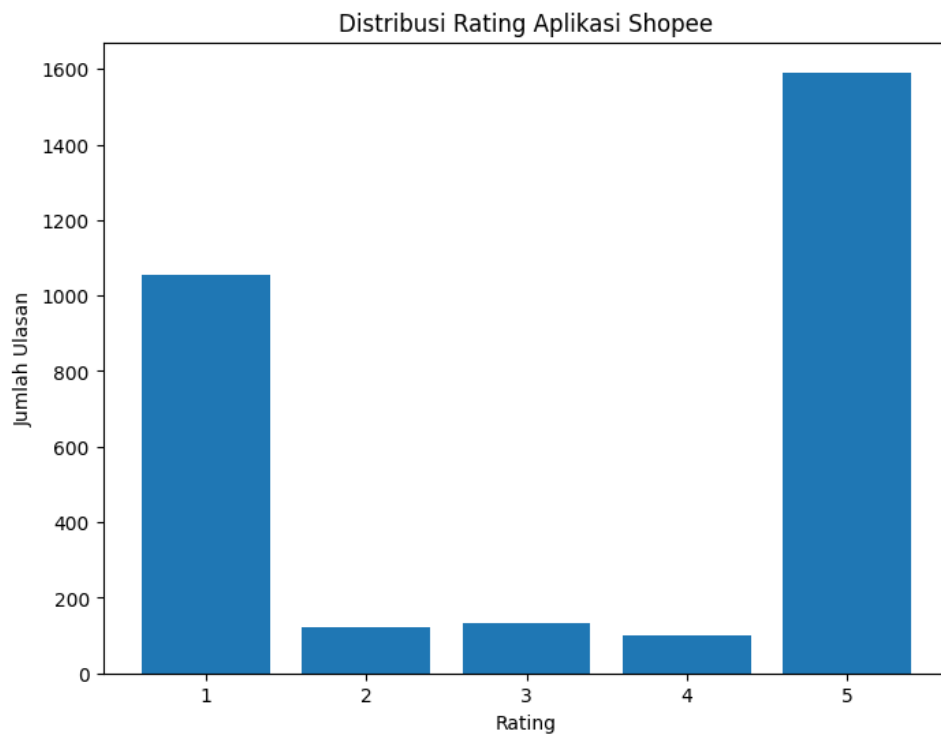
3.1 Ringkasan Data Hasil Scraping

Total 3.000 ulasan berhasil dikumpulkan dari Shopee Indonesia. Setelah diseleksi menjadi 9 kolom relevan dan diberi nama Bahasa Indonesia, data siap untuk tahap eksplorasi. Ringkasan statistik deskriptif menunjukkan rata-rata rating 3,35 dari skala 1–5, dengan jumlah like rata-rata 0,76 per ulasan (maksimum mencapai 1.061 like pada satu ulasan viral).

3.2 Distribusi Rating

Distribusi rating menunjukkan pola bimodal (dua puncak) yang khas pada ulasan aplikasi berskala besar pengguna cenderung memberi rating ekstrem (sangat puas atau sangat kecewa) dibanding rating di tengah.

Rating	Jumlah Ulasan	Persentase
1 Bintang	1.054	35,1%
2 Bintang	122	4,1%
3 Bintang (netral, dibuang)	134	4,5%
4 Bintang	99	3,3%
5 Bintang	1.591	53,0%

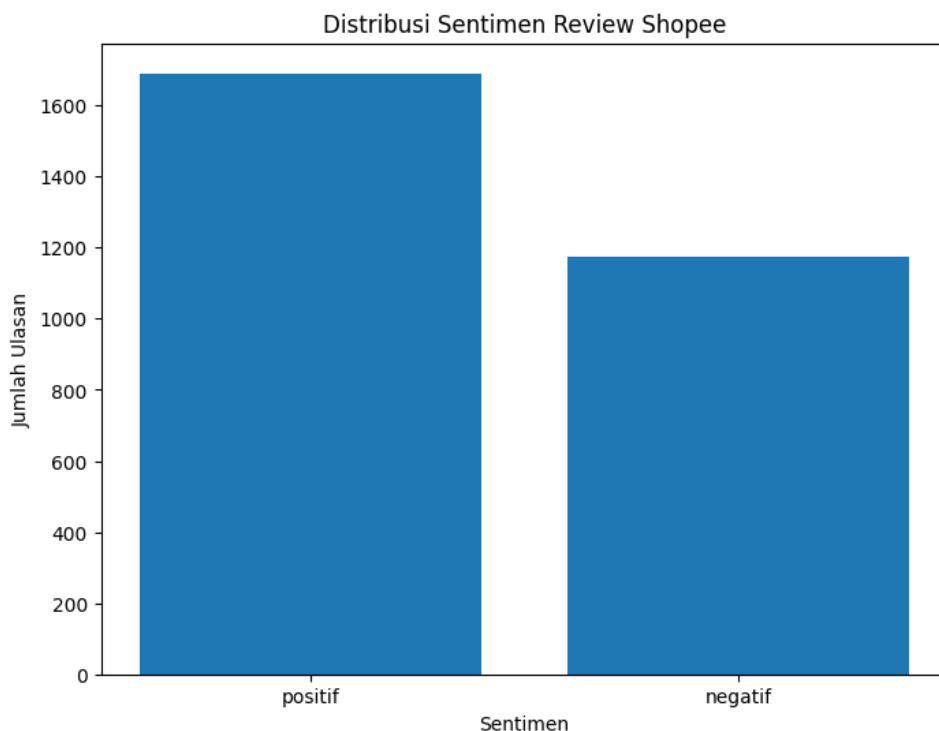


Grafik ini menjadi dasar keputusan pelabelan sentimen pada tahap berikutnya: pola bimodal (dominasi bintang 1 dan 5) mengonfirmasi bahwa rating 3 memang jarang dan wajar untuk dikeluarkan dari data

pemodelan tanpa kehilangan banyak informasi, sekaligus mengindikasikan pengguna aplikasi cenderung memberi rating ekstrem saat sangat puas atau sangat kecewa.

3.3 Distribusi Sentimen

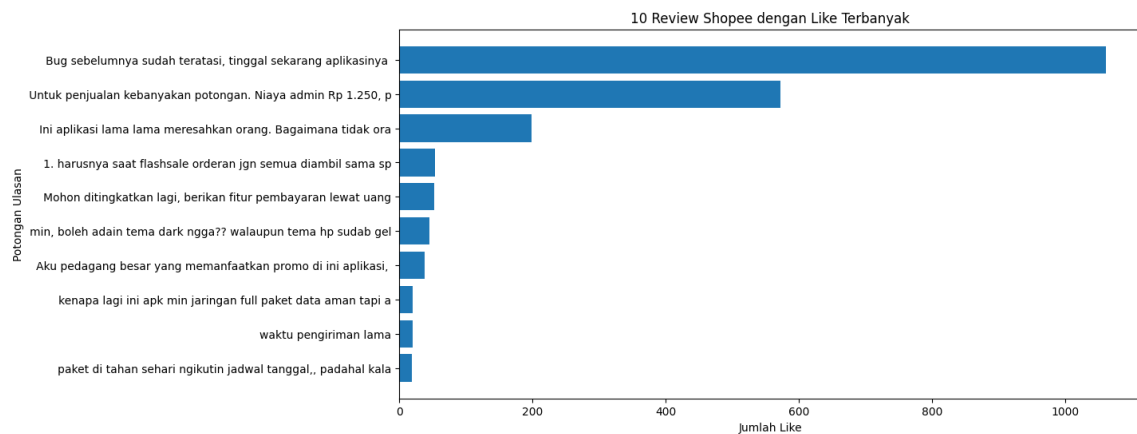
Setelah rating netral (bintang 3) dikeluarkan, data pemodelan berjumlah 2.866 ulasan dengan komposisi 1.690 ulasan positif (59%) dan 1.176 ulasan negatif (41%). Ketimpangan kelas ini relatif ringan dan sudah diantisipasi melalui penggunaan metrik weighted average serta parameter stratify pada proses train-test split, sehingga proporsi kelas tetap terjaga di setiap subset data.



Visualisasi ini dipakai untuk memeriksa keseimbangan kelas (class balance) sebelum pemodelan. Komposisi 59% positif berbanding 41% negatif tergolong cukup seimbang, sehingga metrik accuracy pada laporan ini masih dapat diandalkan sebagai indikator performa tanpa perlu teknik penyeimbangan data tambahan seperti oversampling.

3.4 Ulasan dengan Engagement Tertinggi

Ulasan dengan jumlah like terbanyak umumnya mencerminkan keluhan atau apresiasi yang paling banyak digaungkan oleh komunitas pengguna sinyal kuat untuk memprioritaskan isu yang perlu ditindaklanjuti tim produk.

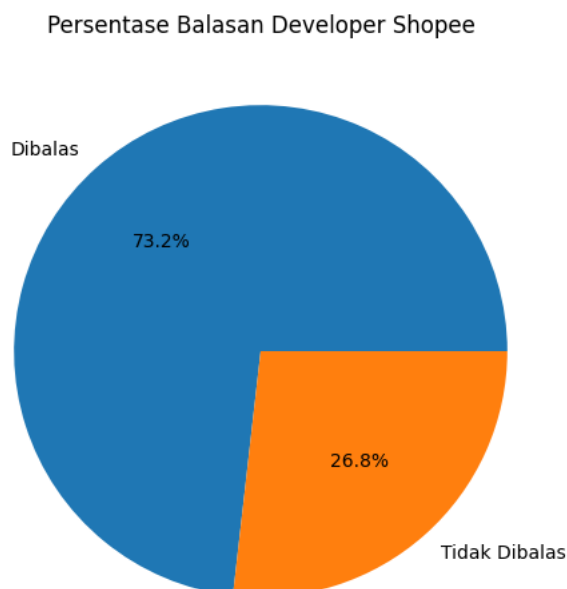


Tabel ini berfungsi sebagai pengecekan kualitatif (qualitative sanity-check) di luar angka statistik semata melihat langsung isi ulasan yang paling banyak dianggap membantu oleh sesama pengguna membantu memvalidasi apakah tema keluhan/apresiasi yang ditemukan lewat word cloud dan analisis sentimen benar-benar mencerminkan pengalaman nyata pengguna.

Menariknya, ulasan dengan like tertinggi (1.061 like, rating 5 bintang) justru melaporkan penyelesaian bug yang sebelumnya bermasalah menunjukkan bahwa transparansi dan resolusi masalah teknis sangat dihargai komunitas pengguna.

3.5 Analisis Respons Developer

Dari total ulasan yang dianalisis, 2.098 ulasan (73,2%) telah mendapat balasan dari tim developer Shopee, sementara 768 ulasan (26,8%) belum direspons.



Fungsi & interpretasi: Pie chart ini relevan untuk menilai kualitas layanan pelanggan (customer service) Shopee secara kuantitatif, sekaligus menjadi dasar rekomendasi bisnis pada Bab 6 mengenai prioritas pembalasan ulasan yang belum direspons.

Word cloud ini menjadi bukti visual bahwa pipeline preprocessing (cleaning, normalisasi, stopwords removal, stemming) berhasil menyisakan kata-kata bermuatan makna, sehingga dapat langsung dipakai untuk mengidentifikasi topik dominan tanpa distraksi kata fungsi seperti pada Gambar 3.5.



Word cloud versi ulasan negatif ini secara khusus difungsikan untuk mengarahkan rekomendasi bisnis pada Bab 6, kata-kata seperti “iklan” dan “paket” yang menonjol menjadi dasar kuantitatif untuk memprioritaskan perbaikan pada aspek iklan dan pengiriman, bukan sekadar opini subjektif penulis laporan.

Insight dari Word Cloud

Dua tema keluhan paling menonjol pada ulasan negatif adalah (1) iklan yang dianggap mengganggu pengalaman berbelanja, dan (2) performa aplikasi yang terasa berat/lambat. Kedua isu ini bersifat dapat ditindaklanjuti langsung oleh tim produk tanpa memerlukan riset tambahan.

3.7 Hasil Evaluasi Model

Tabel berikut merangkum hasil lengkap seluruh 25 kombinasi (5 model x 5 skema split), diurutkan berdasarkan skema split lalu nama model.

```
import pandas as pd
hasil_df = pd.DataFrame(hasil)
```

	Split	Model	Accuracy	Precision	Recall	F1-Score
0	80:20	Naive Bayes	0.904181	0.904247	0.904181	0.904211
1	80:20	Decision Tree	0.843206	0.842673	0.843206	0.842771
2	80:20	KNN	0.641115	0.734804	0.641115	0.552097
3	80:20	Random Forest	0.888502	0.888863	0.888502	0.888638
4	80:20	Logistic Regression	0.895470	0.895246	0.895470	0.895257
5	75:25	Naive Bayes	0.914923	0.915085	0.914923	0.914988
6	75:25	Decision Tree	0.839609	0.838980	0.839609	0.839010
7	75:25	KNN	0.635983	0.735707	0.635983	0.540426
8	75:25	Random Forest	0.899582	0.899840	0.899582	0.899682
9	75:25	Logistic Regression	0.903766	0.903574	0.903766	0.903581
10	70:30	Naive Bayes	0.910465	0.910845	0.910465	0.910593
11	70:30	Decision Tree	0.845349	0.844975	0.845349	0.845105
12	70:30	KNN	0.638372	0.711428	0.638372	0.551904
13	70:30	Random Forest	0.893023	0.893377	0.893023	0.893156
14	70:30	Logistic Regression	0.897674	0.897595	0.897674	0.897630
15	65:35	Naive Bayes	0.909363	0.909478	0.909363	0.909412
16	65:35	Decision Tree	0.859562	0.859116	0.859562	0.858949
17	65:35	KNN	0.633466	0.725463	0.633466	0.537613
18	65:35	Random Forest	0.893426	0.893229	0.893426	0.893283
19	65:35	Logistic Regression	0.894422	0.894185	0.894422	0.894172
20	60:40	Naive Bayes	0.908457	0.908636	0.908457	0.908529
21	60:40	Decision Tree	0.843069	0.842487	0.843069	0.842257
22	60:40	KNN	0.636443	0.733781	0.636443	0.542272
23	60:40	Random Forest	0.890148	0.890082	0.890148	0.890112
24	60:40	Logistic Regression	0.896251	0.896028	0.896251	0.895981

Berikut table versi rapihnya

Split	Model	Accuracy	Precision	Recall	F1-Score
80:20	Naive Bayes	0.9042	0.9042	0.9042	0.9042
80:20	Decision Tree	0.8432	0.8427	0.8432	0.8428
80:20	KNN	0.6411	0.7348	0.6411	0.5521
80:20	Random Forest	0.8885	0.8889	0.8885	0.8886
80:20	Logistic Regression	0.8955	0.8952	0.8955	0.8953
75:25	Naive Bayes	0.9149	0.9151	0.9149	0.9150
75:25	Decision Tree	0.8396	0.8390	0.8396	0.8390
75:25	KNN	0.6360	0.7357	0.6360	0.5404
75:25	Random Forest	0.8996	0.8998	0.8996	0.8997
75:25	Logistic Regression	0.9038	0.9036	0.9038	0.9036
70:30	Naive Bayes	0.9105	0.9108	0.9105	0.9106
70:30	Decision Tree	0.8453	0.8450	0.8453	0.8451
70:30	KNN	0.6384	0.7114	0.6384	0.5519
70:30	Random Forest	0.8930	0.8934	0.8930	0.8932

Split	Model	Accuracy	Precision	Recall	F1-Score
70:30	Logistic Regression	0.8977	0.8976	0.8977	0.8976
65:35	Naive Bayes	0.9094	0.9095	0.9094	0.9094
65:35	Decision Tree	0.8596	0.8591	0.8596	0.8589
65:35	KNN	0.6335	0.7255	0.6335	0.5376
65:35	Random Forest	0.8934	0.8932	0.8934	0.8933
65:35	Logistic Regression	0.8944	0.8942	0.8944	0.8942
60:40	Naive Bayes	0.9085	0.9086	0.9085	0.9085
60:40	Decision Tree	0.8431	0.8425	0.8431	0.8423
60:40	KNN	0.6364	0.7338	0.6364	0.5423
60:40	Random Forest	0.8901	0.8901	0.8901	0.8901
60:40	Logistic Regression	0.8963	0.8960	0.8963	0.8960

3.8 Ranking Global, 10 Kombinasi Terbaik

Ranking diurutkan berdasarkan F1-Score sebagai metrik utama, karena metrik ini menyeimbangkan precision dan recall secara adil.

```

ranking = hasil_df.sort_values(
    by="Accuracy",
    ascending=False
).reset_index(drop=True)

ranking.round(4)

```

	Split	Model	Accuracy	Precision	Recall	F1-Score
0	75:25	Naive Bayes	0.9149	0.9151	0.9149	0.9150
1	70:30	Naive Bayes	0.9105	0.9108	0.9105	0.9106
2	65:35	Naive Bayes	0.9094	0.9095	0.9094	0.9094
3	60:40	Naive Bayes	0.9085	0.9086	0.9085	0.9085
4	80:20	Naive Bayes	0.9042	0.9042	0.9042	0.9042
5	75:25	Logistic Regression	0.9038	0.9036	0.9038	0.9036
6	75:25	Random Forest	0.8996	0.8998	0.8996	0.8997
7	70:30	Logistic Regression	0.8977	0.8976	0.8977	0.8976
8	60:40	Logistic Regression	0.8963	0.8960	0.8963	0.8960
9	80:20	Logistic Regression	0.8955	0.8952	0.8955	0.8953
10	65:35	Logistic Regression	0.8944	0.8942	0.8944	0.8942
11	65:35	Random Forest	0.8934	0.8932	0.8934	0.8933
12	70:30	Random Forest	0.8930	0.8934	0.8930	0.8932
13	60:40	Random Forest	0.8901	0.8901	0.8901	0.8901
14	80:20	Random Forest	0.8885	0.8889	0.8885	0.8886
15	65:35	Decision Tree	0.8596	0.8591	0.8596	0.8589
16	70:30	Decision Tree	0.8453	0.8450	0.8453	0.8451
17	80:20	Decision Tree	0.8432	0.8427	0.8432	0.8428
18	60:40	Decision Tree	0.8431	0.8425	0.8431	0.8423
19	75:25	Decision Tree	0.8396	0.8390	0.8396	0.8390
20	80:20	KNN	0.6411	0.7348	0.6411	0.5521
21	70:30	KNN	0.6384	0.7114	0.6384	0.5519
22	60:40	KNN	0.6384	0.7338	0.6384	0.5423
23	75:25	KNN	0.6360	0.7357	0.6360	0.5404
24	65:35	KNN	0.6335	0.7255	0.6335	0.5376

Berikut table versi rapih dan mudah dibaca

Rank	Split	Model	Accuracy	F1-Score
1	75:25	Naive Bayes	0.9149	0.9150
2	70:30	Naive Bayes	0.9105	0.9106
3	65:35	Naive Bayes	0.9094	0.9094
4	60:40	Naive Bayes	0.9085	0.9085
5	80:20	Naive Bayes	0.9042	0.9042
6	75:25	Logistic Regression	0.9038	0.9036
7	75:25	Random Forest	0.8996	0.8997
8	70:30	Logistic Regression	0.8977	0.8976
9	60:40	Logistic Regression	0.8963	0.8960
10	80:20	Logistic Regression	0.8955	0.8953

Kombinasi Terbaik Secara Keseluruhan

Naive Bayes pada skema split 75:25

Accuracy 91,49% | Precision 91,51% | Recall 91,49% | F1-Score 91,50%.

3.9 Split Terbaik untuk Setiap Model

Selain ranking global, penting juga melihat skema split optimal untuk masing-masing algoritma secara individual, berguna bila tim ingin tetap menggunakan model tertentu (misalnya karena alasan interpretabilitas) namun tetap mendapat performa terbaiknya.

```

2. Split Terbaik untuk Setiap Model

# Ambil baris dengan F1-Score tertinggi untuk masing-masing model
best_split_per_model = (
    hasil_df.sort_values(metric_utama, ascending=False)
    .groupby("Model", as_index=False)
    .first()
    .sort_values(metric_utama, ascending=False)
    .reset_index(drop=True)
)

best_split_per_model.insert(0, "Rank", best_split_per_model.index + 1)

print("=== SPLIT TERBAIK UNTUK SETIAP MODEL ===")
display(best_split_per_model[["Rank", "Model", "Split", "Accuracy", "Precision", "Recall", "F1-Score"]])

```

Rank	Model	Split	Accuracy	Precision	Recall	F1-Score
0	1	Naive Bayes	75:25	0.9149	0.9151	0.9149
1	2	Logistic Regression	75:25	0.9038	0.9036	0.9038
2	3	Random Forest	75:25	0.8996	0.8998	0.8996
3	4	Decision Tree	65:35	0.8596	0.8591	0.8596
4	5	KNN	80:20	0.6411	0.7348	0.6411

Rank	Model	Split Terbaik	Accuracy	F1-Score
1	Naive Bayes	75:25	0.9149	0.9150
2	Logistic Regression	75:25	0.9038	0.9036
3	Random Forest	75:25	0.8996	0.8997
4	Decision Tree	65:35	0.8596	0.8589
5	KNN	80:20	0.6411	0.5521

3.10 Konsistensi Model, Rata-Rata di Seluruh Skema Split

Untuk menilai stabilitas model (bukan hanya performa puncaknya), berikut rata-rata metrik masing-masing model di seluruh 5 skema split yang diuji.

```

3. Ranking Model Berdasarkan Rata-Rata Semua Split

Untuk melihat model mana yang paling stabil/konsisten bagus di semua proporsi split, bukan cuma di satu split saja.

model_avg = (
    hasil_df.groupby("Model")[["Accuracy", "Precision", "Recall", "F1-Score"]]
    .mean()
    .round(4)
    .sort_values("F1-Score", ascending=False)
    .reset_index()
)

model_avg.insert(0, "Rank", model_avg.index + 1)

print("=== RANKING MODEL (rata-rata seluruh split) ===")
display(model_avg)

```

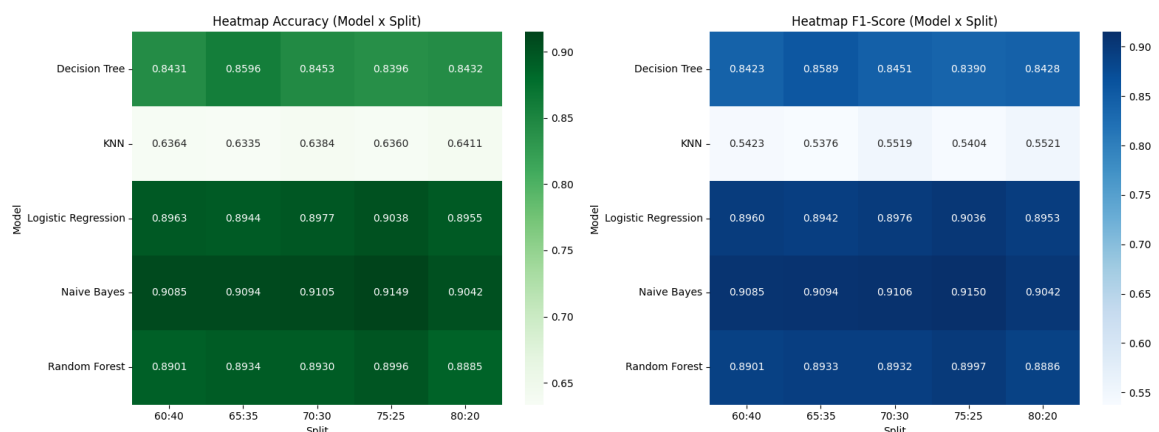
Rank	Model	Accuracy	Precision	Recall	F1-Score
0	1	Naive Bayes	0.9095	0.9096	0.9095
1	2	Logistic Regression	0.8975	0.8973	0.8975
2	3	Random Forest	0.8929	0.8931	0.8929
3	4	Decision Tree	0.8462	0.8457	0.8462
4	5	KNN	0.6371	0.7282	0.6371

Rank	Model	Rata-Rata Accuracy	Rata-Rata Precision	Rata-Rata Recall	Rata-Rata F1-Score
1	Naive Bayes	0.9095	0.9096	0.9095	0.9095
2	Logistic Regression	0.8975	0.8973	0.8975	0.8973
3	Random Forest	0.8929	0.8931	0.8929	0.8930
4	Decision Tree	0.8462	0.8457	0.8462	0.8456
5	KNN	0.6371	0.7282	0.6371	0.5449

Naive Bayes tetap menempati peringkat pertama secara rata-rata (F1-Score 90,95%), mengonfirmasi bahwa keunggulannya bukan kebetulan dari satu skema split saja, melainkan konsisten di seluruh variasi proporsi data latih/uji yang diuji.

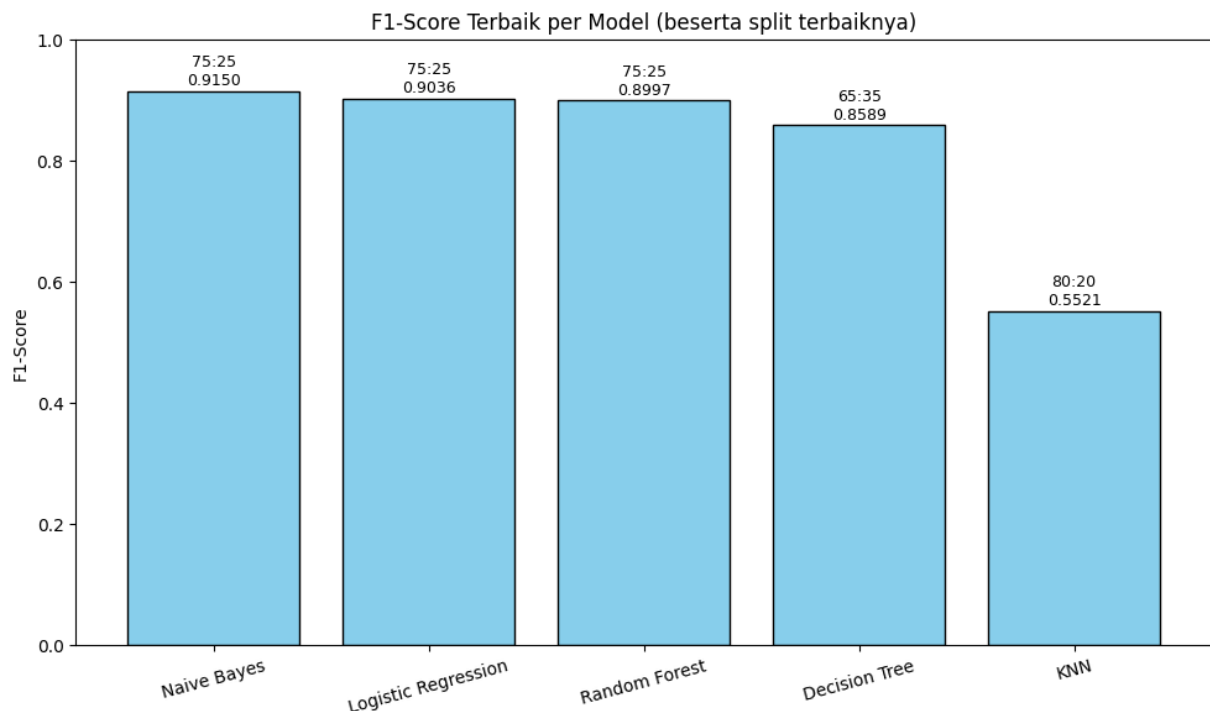
3.11 Visualisasi Perbandingan Model x Split

Heatmap berikut memvisualisasikan seluruh 25 kombinasi hasil secara sekaligus, mempermudah identifikasi pola secara visual: warna yang lebih pekat menunjukkan performa yang lebih tinggi.



Heatmap dipilih sebagai bentuk visualisasi karena mampu menyampaikan 25 angka sekaligus dalam satu pandangan mata, gradasi warna memungkinkan langsung mengenali pola (baris Naive Bayes yang konsisten gelap/tinggi) tanpa perlu membaca satu per satu angka di tabel, mempercepat proses interpretasi bagi pembaca non-teknis sekalipun.

Pola pada heatmap memperkuat temuan sebelumnya: baris Naive Bayes secara konsisten paling gelap (skor tertinggi) di seluruh kolom split, sementara baris KNN secara konsisten paling terang (skor terendah) mengindikasikan KNN kurang cocok untuk data teks berdimensi tinggi seperti representasi TF-IDF pada kasus ini.



Grafik batang ini merangkum seluruh analisis model+split menjadi satu rekomendasi visual tunggal: memudahkan pengambil keputusan untuk langsung melihat model dan skema split mana yang sebaiknya dipakai, tanpa perlu membaca ulang seluruh tabel 25 baris.

3.12 Confusion Matrix

```

7. Confusion Matrix per Model (di Split Terbaiknya)

[]
# CONFUSION MATRIX untuk setiap model, di split terbaiknya masing-masing
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for i, row in best_split_per_model.iterrows():
    model_name = row["Model"]
    split_name = row["Split"]
    test_size = split_list[split_name]

    # Split ulang sesuai split terbaik model ini
    X_train_text, X_test_text, y_train, y_test = train_test_split(
        X, y,
        test_size=test_size,
        random_state=42,
        stratify=y
    )

    # TF-IDF
    tfidf = TfidfVectorizer()
    X_train = tfidf.fit_transform(X_train_text)
    X_test = tfidf.transform(X_test_text)

    # Training ulang model (fresh) di split terbaiknya
    model = models[model_name]
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    cm = confusion_matrix(y_test, y_pred, labels=label_encoder.classes_)

    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=label_encoder.classes_,
        yticklabels=label_encoder.classes_,
        ax=axes[i]
    )

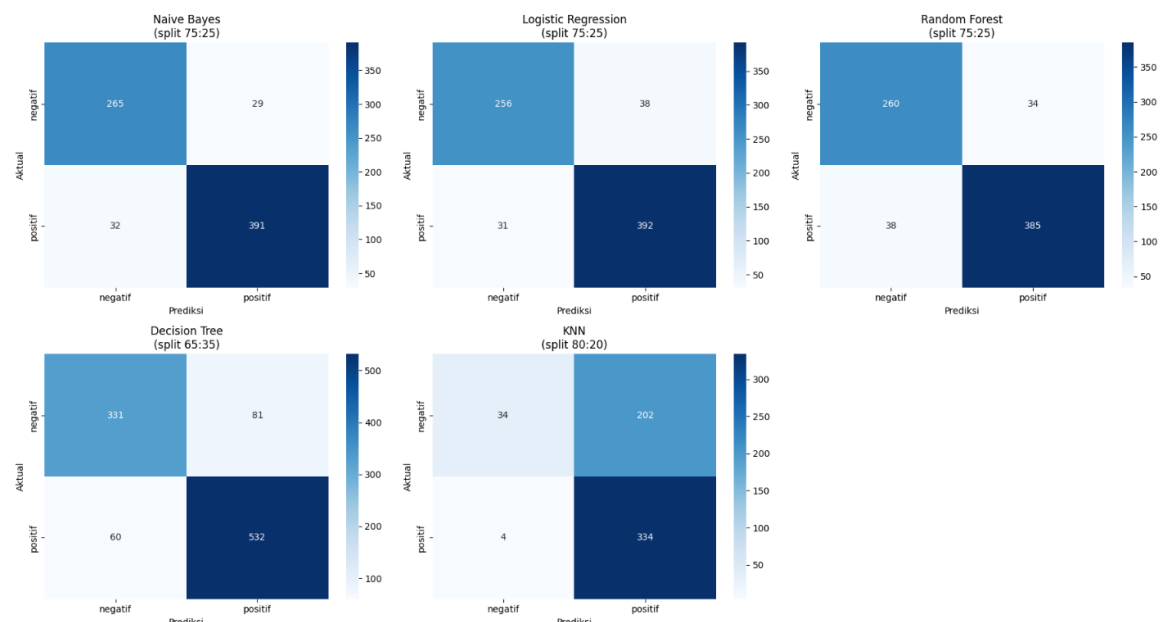
    axes[i].set_title(f"{model_name}\n(split {split_name})")
    axes[i].set_xlabel("Prediksi")
    axes[i].set_ylabel("Aktual")
  
```

Pada tahap ini, dibuat confusion matrix untuk setiap algoritma menggunakan skenario train-test split terbaik yang diperoleh dari hasil evaluasi sebelumnya. Dataset kembali dibagi, ditransformasikan menggunakan TF-IDF, kemudian model dilatih ulang agar menghasilkan prediksi pada data uji.

Selanjutnya, fungsi `confusion_matrix()` digunakan untuk menghitung jumlah prediksi yang benar maupun salah, sedangkan `seaborn.heatmap()` digunakan untuk memvisualisasikannya dalam bentuk heatmap. Visualisasi ini memudahkan analisis kemampuan setiap model dalam mengklasifikasikan sentimen positif dan negatif, sehingga dapat diketahui pola kesalahan prediksi yang dihasilkan oleh masing-masing algoritma.

Confusion matrix digunakan untuk mengetahui jumlah prediksi yang benar maupun salah dari setiap model klasifikasi. Pada matriks ini terdapat empat kemungkinan hasil prediksi, yaitu:

- True Positive (TP) adalah jumlah data sentimen positif yang berhasil diprediksi sebagai positif.
- True Negative (TN) adalah jumlah data sentimen negatif yang berhasil diprediksi sebagai negatif.
- False Positive (FP) adalah jumlah data sentimen negatif yang salah diprediksi sebagai positif.
- False Negative (FN) adalah jumlah data sentimen positif yang salah diprediksi sebagai negatif.



Berdasarkan hasil confusion matrix yang diperoleh:

Model	TN	FP	FN	TP
Naïve Bayes	265	29	32	391
Logistic Regression	256	38	31	392
Random Forest	260	34	38	385
Decision Tree	331	81	60	532
KNN	34	202	4	334

Dari hasil tersebut dapat diketahui bahwa Naïve Bayes, Logistic Regression, dan Random Forest memiliki jumlah prediksi benar (TP dan TN) yang tinggi dengan kesalahan prediksi (FP dan FN) yang relatif rendah. Decision Tree juga mampu mengklasifikasikan sebagian besar data dengan benar, namun masih menghasilkan kesalahan prediksi yang lebih besar dibandingkan ketiga model tersebut. Sementara itu, KNN memiliki nilai False Positive yang sangat tinggi, yaitu sebanyak 202 data, sehingga banyak ulasan negatif

yang diprediksi sebagai positif. Hal ini menunjukkan bahwa performa KNN lebih rendah dibandingkan model lainnya dalam membedakan kedua kelas sentimen.

```

8. Confusion Matrix Model Terbaik Secara Keseluruhan

# CONFUSION MATRIX untuk model TERBAIK secara keseluruhan
best_model_name = best_overall["Model"]
best_split_name = best_overall["Split"]
best_test_size = split_list[best_split_name]

X_train_text, X_test_text, y_train, y_test = train_test_split(
    X, y,
    test_size=best_test_size,
    random_state=42,
    stratify=y
)

tfidf = TfidfVectorizer()
X_train = tfidf.fit_transform(X_train_text)
X_test = tfidf.transform(X_test_text)

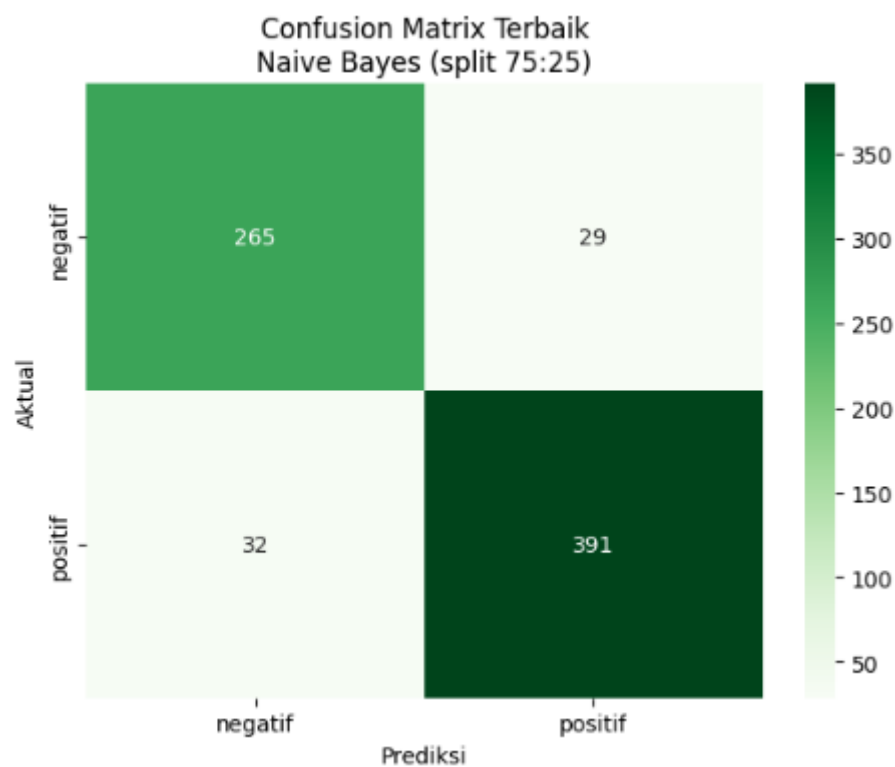
best_model = models[best_model_name]
best_model.fit(X_train, y_train)
y_pred = best_model.predict(X_test)

cm = confusion_matrix(y_test, y_pred, labels=label_encoder.classes_)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_
)
plt.title(f"Confusion Matrix Terbaik\n(best_model_name) (split {best_split_name})")
plt.xlabel("Prediksi")
plt.ylabel("Aktual")
plt.tight_layout()
plt.show()

print("Classification Report:\n")
print(classification_report(y_test, y_pred, target_names=label_encoder.classes_))

```



4. Temuan Kunci

4.1 Temuan Teknis (Model & Data)

- Naive Bayes adalah model paling optimal dan paling stabil untuk kasus ini, unggul di seluruh 5 skema split yang diuji cocok sebagai baseline produksi karena juga ringan secara komputasi dibanding Random Forest maupun model ensemble lain.
- Logistic Regression menjadi runner-up yang solid (F1-Score rata-rata 89,73%), dapat menjadi kandidat alternatif bila dibutuhkan interpretabilitas koefisien fitur.
- KNN adalah model paling tidak cocok untuk kasus ini (F1-Score rata-rata hanya 54,49%), hal ini wajar karena KNN cenderung kurang efektif pada data berdimensi tinggi dan sparse seperti representasi TF-IDF.
- Skema split 75:25 secara konsisten menghasilkan performa terbaik untuk 3 dari 5 model (Naive Bayes, Logistic Regression, Random Forest), menunjukkan skema ini memberikan keseimbangan optimal antara jumlah data latih dan keandalan evaluasi pada dataset ini.
- Pipeline preprocessing berbahasa Indonesia (Sastrawi + indoNLP + kamus domain e-commerce + whitelist negasi) terbukti krusial tanpa penanganan khusus kata negasi, potensi kesalahan label sentimen akibat stopwords removal cukup signifikan.

4.2 Temuan Bisnis (Konten Ulasan)

- Iklan yang mengganggu adalah keluhan paling menonjol pada ulasan negatif berpotensi memengaruhi persepsi pengguna terhadap kualitas aplikasi secara keseluruhan, meski secara fungsional aplikasi berjalan normal.
- Performa aplikasi (berat/lambat, gagal dijalankan) menjadi keluhan teknis kedua yang paling sering disebut.
- Isu pengiriman (waktu pengiriman lama, paket ditahan) tetap menjadi sumber ketidakpuasan yang berulang, konsisten dengan temuan pada ulasan dengan like tertinggi.
- Ulasan positif didominasi apresiasi terhadap kemudahan belanja, kecepatan layanan, dan keterjangkauan promo aspek yang sebaiknya terus dipertahankan dalam roadmap produk.
- Tingkat respons developer (73,2%) tergolong baik, namun 768 ulasan yang belum dibalas berpotensi menjadi “blind spot” bila tidak diprioritaskan berdasarkan urutan engagement (jumlah like).

5. Risiko & Keterbatasan

Keterbatasan	Dampak	Mitigasi yang Disarankan
Data hanya mencakup ulasan terbaru (sort=NEWEST) pada satu titik waktu	Model dapat kurang merepresentasikan sentimen historis atau musiman	Lakukan scraping berkala (mis. mingguan) untuk membangun dataset time-series
Label sentimen diturunkan dari rating, bukan anotasi manual	Potensi mismatch antara rating dan isi teks (mis. rating 5 tapi teks berisi kritik)	Audit sampel acak secara manual untuk mengukur tingkat kesalahan label
Ulasan rating 3 (netral) dibuang sepenuhnya	Model tidak dapat mendeteksi sentimen netral/campuran	Pertimbangkan skema 3 kelas (positif/netral/negatif) pada iterasi berikutnya
Model berbasis TF-IDF (bag-of-words)	Tidak menangkap konteks kalimat & urutan kata secara mendalam	Eksplorasi model berbasis embedding (IndoBERT) pada iterasi lanjutan
Class imbalance ringan (59% positif vs 41% negatif)	Model berpotensi sedikit bias ke kelas mayoritas	Sudah dimitigasi dengan stratified split & weighted metrics; pantau di produksi

6. Rekomendasi

6.1 Rekomendasi Teknis

1. Deploy model Naive Bayes (split 75:25) sebagai baseline produksi untuk sistem monitoring sentimen otomatis model ini ringan, cepat dilatih ulang, dan performanya terbukti paling stabil.
2. Bangun pipeline retraining berkala (mis. bulanan) dengan data ulasan terbaru, agar model tetap relevan mengikuti perubahan gaya bahasa dan tren keluhan pengguna.
3. Pertimbangkan eksplorasi model berbasis deep learning (IndoBERT / transformer Bahasa Indonesia) pada iterasi berikutnya untuk menangkap konteks kalimat yang lebih kaya, terutama untuk kasus sarkasme atau kalimat majemuk.
4. Tambahkan skema klasifikasi 3 kelas (positif/netral/negatif) agar ulasan rating 3 tidak terbuang percuma dan dapat memberi sinyal “waspada dini” sebelum sentimen memburuk.
5. Bangun dashboard monitoring sentimen real-time yang mengonsumsi output model ini, dilengkapi drill-down ke topik keluhan (iklan, performa, pengiriman) agar tim produk dapat memantau tren dari waktu ke waktu.

6.2 Rekomendasi Bisnis

1. Evaluasi frekuensi dan penempatan iklan dalam aplikasi keluhan terkait iklan adalah tema paling dominan pada ulasan negatif dan berisiko menurunkan persepsi kualitas aplikasi secara keseluruhan.
2. Prioritaskan optimasi performa aplikasi (kecepatan buka aplikasi, penggunaan memori) khususnya pada perangkat kelas menengah-bawah yang mendominasi basis pengguna Indonesia.
3. Lakukan audit proses pengiriman & estimasi waktu tiba, karena isu ini secara konsisten muncul pada ulasan dengan engagement (like) tertinggi mengindikasikan dampaknya dirasakan banyak pengguna.
4. Prioritaskan pembalasan pada 768 ulasan yang belum direspons, diurutkan berdasarkan jumlah like tertinggi terlebih dahulu, untuk memaksimalkan dampak persepsi positif dari upaya customer service.
5. Gunakan hasil klasifikasi sentimen sebagai salah satu KPI kualitatif pelengkap rating bintang dalam laporan performa aplikasi bulanan ke manajemen.